

Computational Biomathematics: Collective Behavior, AI-Assisted Coding & Testing

Prof. Dr. Dr. h.c. Florian Rupp
Western Caspian University

In agent-based biological modelling, striking global behavior often emerges from very simple local rules. But scientific insight requires more than visual appeal: it also requires clear assumptions, careful implementation, and tests that check whether the code behaves as intended.

In this lab, we explore how complex collective behavior can emerge from simple local interaction rules. Many biological systems — from bacteria to fish to ants to animal herds — exhibit coordinated behavior without any central controller. This makes them ideal examples for agent-based modelling.

In all four settings of this lab, the main idea is the same: each agent follows only simple local rules, yet the whole population may display large-scale structure and coordination. This is one of the central ideas of modern mathematical biology.

At the same time, this lab also introduces an important aspect of scientific programming: **testing**. A simulation is not scientifically useful merely because it runs and produces a nice picture. We must also ask whether the code behaves in a plausible and consistent way. For this reason, you will use AI not only to help write code, but also to help design `pytest` test cases that check important properties of your model.

A scientifically meaningful simulation combines three ingredients: a biological idea, a computational implementation, and tests that check whether important properties hold.

General Structure of the Models

All groups work with moving agents in two-dimensional space. Each agent has a position and a velocity. In the simplest form, one time step of the simulation looks like this:

```
positions = positions + velocities * dt
velocities = update_velocity(positions, velocities)
```

The details of the update rule differ between the groups. In some models, agents react to neighbors. In others, they react to a field in the environment or to a target.

General Workflow

Please proceed in the following order:

1. Read the biological story of your group carefully.
2. Think about the simplest local rules that could reproduce the behavior.
3. Design a prompt for AI to generate a modular Python implementation.
4. Implement and run the simulation.
5. Think about what should always be true in your model.
6. Use AI to help formulate pytest test cases.
7. Run the tests and improve your code if necessary.

Hint: Do not try to make the model too complicated. A small, clear, and testable model is much better than a complicated one that nobody fully understands.

A Small Common Starter Structure

The following minimal code skeleton may be useful for all groups.

```
import numpy as np
import matplotlib.pyplot as plt

def initialize_agents(n):
    positions = np.random.rand(n, 2)
    velocities = 0.01 * np.random.randn(n, 2)
    return positions, velocities

def step(positions, velocities, dt=1.0):
    velocities = update_velocity(positions, velocities)
    positions = positions + dt * velocities
    return positions, velocities
```

You may also find it useful to keep agents inside the domain $[0,1] \times [0,1]$. One simple approach is clipping:

```
positions = np.clip(positions, 0.0, 1.0)
```

Another possible approach is periodic boundaries. You may choose either method, but be explicit in your code.

Group A: Chemotaxis

Chemotaxis is the directed movement of cells or microorganisms in response to a chemical signal. Many bacteria, for example, are able to detect gradients of nutrients in their environment and move toward regions of higher concentration. Similarly, immune cells in the human body move toward sites of infection by following chemical signals released by damaged tissue.

At first glance, this behavior appears purposeful, almost as if the cells “know” where to go. However, at the microscopic level, the movement is much simpler: cells alternate between random motion and slight directional biases depending on whether the chemical concentration increases or decreases.

In your model, you should think of each agent as having only very limited information. It can sense something about its local environment, for example whether it is moving toward or away from a source, but it does not know the full structure of the chemical field.

A useful starting point is to combine two components: random movement, which models uncertainty and noise, and a small directional bias toward higher concentration. Even such a simple rule can already lead to accumulation near the source.

You can enrich your model by asking questions such as the following. What happens if the signal is weak? What happens if noise is strong? How sensitive must the agents be in order for directed motion to emerge?

Your goal is to understand how a simple local rule can produce the collective effect of accumulation near a source.

Starter Code Snippet

```
def chemical_field(x, y, cx=0.5, cy=0.5):
    return np.exp(-10*((x-cx)**2 + (y-cy)**2))

def update_velocity(pos, vel):
    noise = 0.02 * np.random.randn(*vel.shape)
    grad = np.array([0.5 - pos[:,0], 0.5 - pos[:,1]]).T
    return vel + 0.05 * grad + noise
```

Student Task

Implement a simple chemotaxis model in which agents move under the influence of a chemical source. Start with a source near the center of the domain and let the agents move with a mixture of randomness and directional bias. Visualize how the positions change over time. Once the basic model works, experiment with the strength of the directional bias and the strength of the random noise. Try to describe in words what happens when one of the two effects dominates the other.

AI Prompt Design Task

Write a prompt asking AI to generate a modular Python program for chemotaxis. Your prompt should explicitly ask for

- initialization of agent positions and velocities,
- a chemical concentration field,
- a velocity update with noise and gradient bias,
- visualization of the agent positions over time.

Testing and pytest Hint

Before writing tests, think about which properties should always hold if your code is sensible. Possible properties to test are:

- the shape of the velocity array after updating should remain the same as before,
- all position and velocity values should remain finite,
- if there is no noise and the gradient points to the center, then agents should on average move closer to the center after one or several steps.

A suitable pytest test could therefore check, for example, that the average distance to the source decreases in a simplified deterministic setting.

Group B: Schooling Behavior

Many animals, such as fish and birds, move in highly coordinated groups without any apparent leader. A school of fish can change direction almost instantaneously, and yet each individual follows only simple local rules.

Biological studies suggest that individuals balance three competing tendencies. They avoid collisions with nearby neighbors, they align their direction with others, and they try to remain close to the group. These interactions typically depend only on nearby individuals and not on the entire group.

In your model, each agent represents an individual animal that can observe only its local neighborhood. You should think carefully about how distance influences behavior. Very close neighbors may trigger repulsion in order to avoid collisions, while slightly more distant neighbors may influence alignment, and still more distant ones may create attraction.

A good strategy is to build your model step by step. Start with a very simple rule such as alignment only and observe what happens. Then gradually add additional effects such as repulsion or attraction.

You may also explore how the behavior changes when you vary the interaction radius or the strength of alignment. Under what conditions does the group move as a coherent unit? When does it break apart?

The goal is to see how coordinated group motion can emerge from simple local interactions.

Starter Code Snippet

```
def update_velocity(pos, vel):
    new_vel = vel.copy()
    for i in range(len(pos)):
        diff = pos - pos[i]
        dist = np.linalg.norm(diff, axis=1)

        neighbors = (dist < 0.2) & (dist > 0)
        if np.any(neighbors):
            alignment = vel[neighbors].mean(axis=0)
            new_vel[i] += 0.05 * alignment
    return new_vel
```

Hint: A very good strategy is to implement only alignment first. Once that works, you may add a repulsion rule for very small distances and an attraction rule for somewhat larger distances.

Student Task

Implement a simple schooling model. Begin with alignment and then enrich the rule by adding repulsion and, if time permits, attraction. Visualize the trajectories or the instantaneous positions and directions of the agents. Explore how the collective behavior changes when you modify the neighborhood radius or the strength of the alignment effect.

AI Prompt Design Task

Write a prompt asking AI to implement schooling behavior using local interaction rules. Make sure your prompt explains that you want the model to be built in a simple modular way and that alignment should be implemented first before adding repulsion and attraction.

Testing and pytest Hint

Useful properties to test include:

- the updated velocity array has the correct shape,
- all values remain finite,
- if all agents start with exactly the same velocity and the model contains only alignment, then their velocities should remain equal after the update,
- in a simple test configuration, alignment should reduce the variation in direction between neighboring agents.

For example, you may write a test in which two or three nearby agents have slightly different velocities and check whether the updated velocities become more similar.

Group C: Ant Trail Formation

Ant colonies are capable of constructing efficient foraging networks without any central coordination. When searching for food, ants move more or less randomly. However, once food is found, they begin to deposit pheromones along their path. Other ants are more likely to follow paths with higher pheromone concentration, reinforcing successful routes over time.

This creates a feedback loop: frequently used paths become stronger, while unused paths gradually disappear due to evaporation of pheromones.

In your model, you should consider two interacting components: the agents, which move through space, and the environment, which stores information in the form of a pheromone field. This is different from the other models because communication is indirect: ants do not interact directly with each other, but through the environment.

You can begin with a very simple rule. Let agents perform a random walk, but bias their movement toward higher pheromone concentration. Then update the pheromone field by increasing it where ants move and letting it decay over time.

You may enrich your model by asking the following questions. What happens if pheromones decay quickly? What if they persist for a long time? How does this influence the formation of stable trails?

Your goal is to understand how collective structures — in this case paths or trails — emerge from simple feedback between agents and their environment.

Starter Code Snippet

```
pheromone = np.zeros((100, 100))

def update_pheromone(pheromone):
    pheromone = 0.95 * pheromone
    return pheromone

def deposit_pheromone(pheromone, positions):
    for p in positions:
        i = min(int(p[0] * pheromone.shape[0]), pheromone.shape[0]-1)
        j = min(int(p[1] * pheromone.shape[1]), pheromone.shape[1]-1)
        pheromone[i, j] += 1.0
    return pheromone
```

Student Task

Implement a simple ant trail model in which ants move under the influence of a pheromone field. Start with random movement, then add a bias toward higher pheromone concentration. Let ants deposit pheromone as they move, and let the field decay over time. If the basic model works, explore the influence of the decay rate and the amount of pheromone deposited.

AI Prompt Design Task

Write a prompt asking AI to generate a modular Python simulation of ant trail formation. Your prompt should explicitly mention both components of the model: the moving agents and the pheromone field. It should also ask for clear plotting of both agent positions and, if possible, the pheromone field.

Testing and pytest Hint

A pheromone field suggests several very natural tests:

- pheromone values should remain non-negative,
- if only decay is applied and no new pheromone is deposited, then the total amount of pheromone should decrease,
- depositing pheromone at a position should increase the value of the field at the corresponding grid cell,
- the shape of the pheromone array should remain unchanged after updates.

These are excellent candidates for `pytest`, because they are simple, important, and easy to interpret.

Group D: Crowd Dynamics and Escape Behavior (Savanna Setting)

In many biological systems, large groups of animals move collectively in response to external stimuli such as predators. For example, herds of wildebeest or zebras in the savanna may suddenly change direction and move rapidly toward safer regions when a threat is detected.

Although the movement may appear coordinated, each individual typically reacts only to its immediate surroundings. It tries to move toward safety while also avoiding collisions with nearby animals.

In your model, you can think of each agent as an animal that has two competing objectives: first, to move toward a target, for example a safe region such as water or open terrain; second, to avoid collisions with nearby individuals.

Even with these simple rules, complex collective effects can arise, such as clustering, lane formation, or temporary congestion in narrow regions.

A good starting point is to define a direction toward the target and then add a repulsion term that pushes agents away from very close neighbors. You may then explore how the behavior changes when the density of agents increases or when the strength of repulsion is varied.

You can also ask the following questions. What happens if all individuals strongly follow the target? What happens if avoidance is stronger than attraction? Under which conditions do bottlenecks or crowding effects appear?

Your goal is to understand how large-scale movement patterns arise from simple local decisions in a crowded biological environment.

Starter Code Snippet

```
target = np.array([0.9, 0.5])

def update_velocity(pos, vel):
    direction = target - pos
    norms = np.linalg.norm(direction, axis=1, keepdims=True)
    direction = direction / np.maximum(norms, 1e-8)
```

```

repulsion = np.zeros_like(pos)
for i in range(len(pos)):
    diff = pos[i] - pos
    dist = np.linalg.norm(diff, axis=1)
    close = (dist < 0.08) & (dist > 0)
    if np.any(close):
        repulsion[i] += np.sum(diff[close], axis=0)

return 0.08 * direction + 0.03 * repulsion

```

Student Task

Implement a crowd-dynamics or escape-behavior model in which animals move toward a safe target while avoiding collisions with nearby animals. Visualize how the crowd evolves over time. Once the model works, vary the density of agents or the strength of repulsion and observe whether bottlenecks, clustering, or other large-scale effects appear.

AI Prompt Design Task

Write a prompt asking AI to generate a modular Python program for this model. Your prompt should clearly explain that each agent is pulled toward a target but pushed away from nearby agents, and that the output should include a visualization of the movement over time.

Testing and pytest Hint

Reasonable tests for this model include:

- the output arrays have the correct shape,
- all values remain finite,
- if only the target force is active and there is no repulsion, then the distance to the target should decrease after an update,
- if two agents are extremely close, then the repulsion term should push them apart rather than pull them together.

These are especially good examples of how a test can capture a biological or physical expectation in a very simple mathematical form.

General Advice on pytest

The purpose of testing is not to prove that your code is perfect. Rather, it is to check a few basic properties that should hold if your implementation is sensible. A good test often checks one of the following:

- shapes of arrays,

- absence of NaN or infinite values,
- monotonic effects in a simplified situation,
- non-negativity of physically meaningful quantities,
- preservation of simple symmetries or expected qualitative behavior.

A minimal example could look like this:

```
def test_update_velocity_shape():
    pos = np.array([[0.2, 0.2], [0.4, 0.5]])
    vel = np.array([[0.01, 0.0], [0.0, 0.01]])
    new_vel = update_velocity(pos, vel)
    assert new_vel.shape == vel.shape
```

You can run your tests with

```
pytest
```

Hint: A good rule of thumb is this: if you cannot clearly state what should happen in a simple case, then you probably do not yet fully understand your own model.

Reflection

Student Task

Reflect briefly on the following questions. Was it easy to formulate local rules from the biological story? Did your first AI prompt already produce useful code, or did you have to refine it? Which tests did you choose, and why? Did the tests reveal any mistakes or weaknesses in your implementation? Finally, think about the following larger question: in what sense do your tests increase your confidence in the biological interpretation of the simulation?

Final Presentations

Each group should prepare a short presentation covering the following points:

- the biological system and its local rules,
- the simplest version of the implemented model,
- one or two enrichments of the rule,
- one AI prompt that was particularly useful,
- at least one pytest test case and what it checks,
- one interesting simulation result or visual pattern.